Mini Project #1: Design and Simulation of Industrial Control Systems

CS 6263/ECE 8813: Cyber-Physical Systems Security (Summer 2025)

Assigned: May 14, 2025 **Due:** May 28, 2025, 11:59pm EST

Environment Setup

The first step towards understanding the concepts of cyber-physical systems (CPS) security is to get familiar with their control logics and mechanisms. In the first mini project, we will learn how to model an industrial control system (ICS) with its control logic. To do so, an advanced educational modeling software, namely Factory IO coupled with Control IO, will be used to model the process and design the underlying controller. The latest trial version of Factory IO for Windows machines can be downloaded from here: Factory IO trial version (Alternate Link). You can use this version for 30 days.

Factory IO requires Windows operating system (OS). If you do not have access to a Windows machine, you can download Windows from Georgia Tech Azure Dev Tools (formally Microsoft Imagine). If you have a Mac, then Parallels Desktop is a great way to set up the Windows virtual environment. The trial version of Parallels is sufficient for this mini project but you can also get a student discount for the full version. Otherwise, any virtual machine manager is allowed (e.g., VirtualBox). If you use VirtualBox, then you may need to install Guest Additions, so that GUI resolution auto-adjusts. Please note that Factory IO needs a GPU for better graphical rendering performance and a good amount of RAM. Therefore, make sure you have allocated enough hardware resources from your host machine to get a smooth environment. If you are still unable to get a smooth running experience, try to decrease the graphical rendering quality within the Factory IO settings. (Please avoid using VirtualBox if possible, it is not ideal for development).

Factory IO requires heavy compute resources to run without any inexplicable problems and anomalies. A machine with the following specifications would be ideal for this project. These are not hard requirements, but will help improve your development experience. The listed specifications are based on statistics from previous classes:

- CPU Utilization does not exceed 30%
- 16GB RAM
- 2GB Video RAM
- 50 FPS

Factory IO

Inside Factory IO, there is a visual environment where you can create the 3-D model of an industrial control process. Also, you can design a controller for the process, where the actuators are controlled by sensors, input buttons, and the underlying control logic. The control logic of the process can be embedded inside a programmable logic controller (PLC) connected to your computer. We will learn how to program a PLC in the next mini project. Instead of connecting a real PLC, in this mini project, you will use the Control IO interface with Factory IO to design your controller. You can connect the block diagrams of the inputs, sensors, and actuators with different logic to form your controller. This is very similar to programming in the Simulink/-Matlab environment. Factory IO can be very easy to learn, but for this project you are only working in Control IO.

It does take some time to get familiar and comfortable with Factory IO and Control IO software. This project requires Function Block Diagram design which is a bit different from traditional software programming (there are no IF statements!). Please do not wait till the last weekend to start this project, as the estimated time of completion could be anywhere from 15 to 50 hours. Below are helpful resources to get started on Factory IO, Control IO, PLCs and Function Block Programming.

Table 1: Helpful Resources

Description	**Link**
Free Tutorial	PLC Academy
Control IO Documentation	Getting Started
Factory IO Documentation	Overview
Factory IO Parts Documentation	Parts
Getting started with Control IO	YouTube Video
Summaries of PLC Programming Languages	isd-soft
(Technical Background Section)	Lab Assignment
PLC Design Approach	ACC Automation
Introduction to Function Blocks	Function Blocks

YouTube Videos

To better assist this assignment, a series of YouTube videos have been created to highlight the requirements for each part. Please note, the exact description and implementation in the videos may differ from the project requirements. It is your responsibility to follow the requirements specified in this document. Regardless, these videos should provide a high level understanding of what a working solution looks like.

Table 2: YouTube Videos

Section	**Video**
Water Tank	Link
Pallet Storage	Link
Factory IO General Intro	Link
Factory IO Interface Introduction	Link
RFID Read/Write	Link
RFID Two Readers	Link
CTU/CTD	Link
JK Flip/Flop	Link

Additional Notes

In order to get started and view the Part 1 scene, unzip the attached project zip. Double-click on the scene file to open under the scenes directory (in this case *part1.factoryio*). In turn, this will start Factory IO with the scene loaded.

If Factory IO is opened first, a blank Control IO file might open. If this happens, you should not use this Control IO, instead you will need to open the Control IO file associated with your work. The blank Control IO file will have a file name such as "Diagram1.controlio". That is your indication to load your Control IO file. Make sure that the Control IO file you are editing is the correct file. Do not modify the 3D environment in Factory IO, as the same originally provided scene will be loaded during grading - not the one that you submit. The provided .controlio files contain a block used for a programmatic connection to Canvas for grading and is labeled as such - do not remove or adjust it!

Logic Bombs

A logic bomb is a set of instructions incorporated into a program so that if some condition is met, the instructions will be executed. Typically, triggering a logic bomb causes a harmful change to the normal operation of the program. For the purpose of this assignment, the logic bombs will result in a change to the normal operation, but the change may or may not appear to be harmful to you. However, the plant owner may feel that the unauthorized change is harmful or might result in an unobserved downstream effect that she considers harmful.

1 Water Tank System (35 Points)

Part 1a - Water Tank System (0 Points)

In this part, the goal is to design and implement a water tank system that fills and drains automatically. In the pre-built Factory IO scene, there are three water tanks. However, this part only requires working with one water tank (not all three). Below are the steps required for the implementation:

- The fill process starts when the control box start button is pressed.
- Once the water level reaches 55% of the tank capacity, the fill valve must close and the drain valve must open to decrease the tank level to 35%. This filling and draining process should continue between 35% and 55% until the stop button is pressed.
- By pressing the control box stop button, the remaining water in Tank 1 must begin draining and then the entire process must stop once the tank is empty. Pressing the start button will restart the fill and drain cycle process.
- Pressing the start button while a tank is draining below its minimum cycle level (ex: at 30% for Tank 1) should immediately stop the tank from draining and restart the filling process.

This implementation has already been done and the solution is provided in the assignment zip file. Also, there is a Youtube Video created to discuss how to implement the solution. This is provided as reference and to assist with the learning curve required for working with Factory & Control IO. **Note:** The tank level targets in the YouTube video may differ from the requirements of this write-up. The levels specified in this write-up must be used for your project.

Part 1b - Logic Bomb Water Tank System (35 Points)

This part is an extension to the first and will require using all three water tanks and adding a logic bomb. You may use your part1a.controlio file as a starting point for Part 1b. You should close both Factory IO and Control IO while you copy and rename the file to part1b.controlio and then re-open the program to continue programming.

For Tank 1, once the water level reaches to 55% of the tank capacity, the fill valve must close and the drain valve must open and continue until the water level reaches 35% of tank capacity. At this point, the drain valve must close and the fill valve must open again to refill to 55%. This process must continue working in a repeated cycle until the stop button is pressed. By pressing the stop button, the remaining water in the tank must begin draining and then the entire process must stop once empty.

For Tank 2, once the water level reaches 70% of the tank capacity, the fill valve must close and the drain valve must open and continue until the water level reaches 40% of the tank capacity. This process must continue working in a repeated cycle until the stop button is pressed. By pressing the stop button, the remaining water in the tank must begin draining and then the entire process must stop once empty.

For Tank 3, once the water level reaches 90% of the tank capacity, the fill valve must close and

the drain valve must open and continue until the water level reaches 55% of the tank capacity. This process must continue working in a repeated cycle until the stop button is pressed. By pressing the stop button, the remaining water in the tank must begin draining and then the entire process must stop once empty.

After Tank 2 fills to 70% three times, the fill and drain logic of Tank 1 and 3 should be affected. As a result, your logic bomb should cause the Tank 1 tank to cycle between 14% and 16%. The cycle action is to drain to 14% and when 14% is reached, it fills to 16% and then drains to 14% and continues the 14-16% cycle. Tank 3 should fill to 100% capacity. If the stop button (or start button) is pressed after the logic bomb has triggered, Tank 2 should continue to respond accordingly, however, Tank 1 and 3 should NOT (i.e. ignore the start/stop buttons). Under no conditions shall the count for the logic bomb trigger be reset.

Note: The tank level thresholds described here are nominal levels. As a tank fills and empties, its actual level at any point in the cycle may go just above a threshold, drop below, and then go back above due to tank splashing from the physics of the filling and emptying process. There should be a noticeable and intentional drop in tank level before a high level is considered to be reached for a subsequent cycle. In other words, if a high level threshold is 70%, dropping to 69% and then splashing back up to 70% or higher is not considered to be reaching 70% a second time. There must be an intentional draining of the tank, noticeable drop in level, and refilling of the tank before 70% can be triggered as a high level again.

Grading Notes

A pre-built Factory IO scene is provided and you are not allowed to modify the pre-built scene, as this will be the scene used for grading. Any points lost due to scene modifications are not eligible for a regrade (including any sensors/actuators that are already FORCED). **This part will be graded under normal speed (1x).** If you speed up the scene during programming and testing, ensure it still works at 1x speed! There are no requirements for connecting the different lights and control box displays for this part.

2 Pallet Storage (65 Points)

Part 2a - Pallet Storage (50 Points)

In this section, the goal is to design and implement a pallet sorting and storage system. Below are the steps required for implementation:

- When the control panel start button is pressed, the system shall initiate a 3-second safety delay, with the Warning Light (rotating beacon) activated and the Stack Light illuminating yellow. This safety delay shall activate every time the system is started.
- After the safety delay, the Warning Light deactivates and the Stack Light illuminates green. The emitter will produce pallets with various size boxes: small, medium, and large. The emitter is already configured and activated (forced) for this.
- Once a pallet and box reaches the loading conveyor (pick-off station), the stacker crane shall pick it up and store it in the appropriate rack space. See Appendix B for detail on the various box sizes and where they shall be stored in the rack.

- For each rack section, the first pallet shall be stored beginning in the lower right space in that section and subsequent pallets of that size are stored upward from there. When the top space is filled, begin storing at bottom of the next column to the left. **Note:** Do not worry about filling up an entire section with one size of box. The scene will not be run long enough for that to happen.
- If the stop button is pressed, the process shall pause and continue as normal when the start button is pressed again. The Stacker Crane can continue moving towards its current destination, but it should stop operation once it arrives. If the system is started again, the crane shall either place the pallet it was storing or pick up the next pallet from the pick-off station. **Note:** Do not worry about the stop button being pressed while a pallet is being picked up or placed on the rack, but the stop button could be pressed while the Stacker Crane is in motion to/from the pick-off station.
- Ensure that the control box displays the appropriate number of stored box types. You can utilize or ignore the "Next Location" displays. The configuration is provided in the table below.

Table 3: Part 2 Control Box Configuration

Counters	**When Count is Taken**
Small/Medium/Large Box Inventory	Once pallet is placed on storage rack

Part 2b - Logic Bomb Pallet Storage (15 Points)

For this part, the goal is to introduce a logic bomb into the implementation. You may use your part2a.controlio file as a starting point for Part 2b. You should close both Factory IO and Control IO while you copy and rename the file to part2b.controlio and then re-open the program to continue programming.

The logic bomb should trigger once 12 pallets of any size box have been stored in the rack. In other words, 3 large / 4 medium / 5 small boxes have been stored, or 2 large / 8 medium / 2 small, etc. The logic bomb should cause the Stacker Crane to begin placing pallets where they have already been stored, in effect knocking some pallets off the rack. There should be no changes to the control box inventory display, so it should be reporting incorrect numbers since some of the pallets have been pushed out of inventory. Also, the system shall stop responding to start and stop buttons after the logic bomb has triggered. Under no conditions shall the count for the logic bomb trigger be reset, nor shall the logic bomb effect be cancelled once in place.

Grading Notes

A pre-built Factory IO scene is provided, and you are not allowed to modify the pre-built scene, as this will be the scene used for grading. Any points lost due to scene modifications are not eligible for a regrade (including any sensors/actuators that are already FORCED). **This part will be graded under normal speed (1x).** If you speed up the scene during programming and testing, ensure it still works at 1x speed!

Deliverable and Submission Instructions

Create a zip file named <First Name>-<Last Name>-mp1.zip (e.g., Tohid-Shekari-mp1.zip), that includes all your files and submit it on Canvas. Your zip file should be generated in such a way that when it is extracted, we get two folders with the names of **scenes** and **controllers**. If you make multiple submissions to Canvas, a number will be appended to your filename. This is inserted by Canvas and will not result in any penalty. Part 1a Control IO file was provided as an example and should not be turned in.

Note: Failure to follow the submission and naming instructions will cause 20% points loss.

```
<First Name>-<Last Name>-mp1.zip
|-- scenes
   |-- part1.factoryio
   |-- part2.factoryio
|-- controllers
   |-- part1b.controlio
   |-- part2a.controlio
   |-- part2b.controlio
```

A good way to ensure your scenes being submitted are no different than the original scenes is to compare the original project file hash and your submission file hash. In Windows Powershell, use **Get-FileHash <original scene from zip>** and compare it to **Get-FileHash <your scene for submission>**

Please check your submission multiple times, as any submissions after the deadline will be subject to the late submission penalty per the syllabus. After making your final submission to Canvas, you should download your submission from Canvas and unzip to ensure all files are there and in the correct locations. There will be no exceptions to the late policy for any reason (for example, wrong file submitted, forgot to include a file, etc).

Appendix A: Evaluation

The tables below are a rough estimation of how the TAs will approach grading. It is up to you to read through the requirements of the project and ensure all requirements are satisfied.

Table 4: Part 1 Water Tank Logic Bomb (25 Points)

Requirement	**Points**	**Notes**
Normal Operation	25	All three tanks start filling when Start button pressed, cycle appropriately, begin draining when Stop button pressed, and restart if Start button pressed during or after draining

Table 5: Part 1 Water Tank Logic Bomb (10 Points)

Requirement	**Points**	**Notes**
Modified Operation	5	Tanks operate as described for Logic Bomb
Start/Stop Buttons	5	Start/Stop buttons stop working for specified tanks

Table 6: Part 2 Pallet Storage (50 Points)

Requirement	**Points**	**Notes**
Process Starts/Stops	10	Process starts and stops appropriately when Start/Stop buttons pressed
Pallet Pickup	10	Stacker Crane picks up pallet
Pallet Storage	10	Stacker Crane stores pallet in rack
Organization	10	Stacker Crane stores box sizes in correct section of rack
Control Box Logic	10	Control box numbers are accurately displayed

Table 7: Part 2 Pallet Storage Logic Bomb (15 Points)

Requirement	**Points**	**Notes**
Logic Bomb Trigger	5	Logic bomb triggers after condition has been met
Modified Operation	5	Stacker Crane operates as described for Logic Bomb
Start/Stop Buttons	5	Start/Stop buttons stop working

Extra Credit: To further encourage efficient programming diagrams, bonus points on this project will be granted to five students with the overall lowest block counts. Similar to the penalty thresholds, Notes, Memory Booleans, and Memory Integers will be excluded from this count. The student with the lowest block count will receive 5 additional points on this project. The second lowest will receive 4, third lowest 3, fourth lowest 2, and fifth lowest 1.

In order to receive these points, your project must score perfectly in all sections and you must implement the control box button lights and rotating beacon warning lights where they exist in the scenes. In part 2, the control box button lights should illuminate to indicate the running status of the system (green if running, red if stopped).

You may use the following python code snippet (execute in terminal/IDE) for enumerating the number of blocks in your code:

```
import xml.etree.ElementTree as ET

#Change the file name in the line below:
controlio_path = 'partxx.controlio'

count = 0
ignore_types = ["NotesNode", "InternalMemoryBit", "InternalMemoryInt"]
tree = ET.parse(controlio_path)
root = tree.getroot()
diagram = root[1]
for element in diagram:
  if (element.tag == "NodeData") and (element.attrib["NodeType"] not in ignore_types):
        count += 1
print(count)
```

Appendix B: Pallet Storage

Table 8: Box Sizes

Small Box	Medium Box	Large Box
		

Table 9: Box Size Storage Locations